Java Programming: Exception Handling

Study notes for a document-based Retrieval-Augmented Generation (RAG) project

How to use these notes

These notes provide connected explanations, examples, terminology, and review questions. They are intended as a starter corpus for testing PDF extraction, chunking, embedding, and retrieval. Read the sections and ask questions whose answers appear in the material.

1. What is an exception?

An exception is an event that interrupts the normal flow of a program. Java represents exceptions as objects. Exception handling lets a program detect and respond to exceptional conditions instead of allowing every failure to terminate the application without a useful response.

2. Exception hierarchy

Throwable is the root type for objects that can be thrown. Its main branches are Error and Exception. Errors generally represent serious problems that applications are not expected to recover from. RuntimeException and its subclasses are unchecked exceptions. Other exception classes are checked exceptions.

3. Checked and unchecked exceptions

Checked exceptions must be caught or declared by the method that may throw them, subject to Java's compile-time rules. Examples include IOException. Unchecked exceptions include RuntimeException subclasses such as NullPointerException and IllegalArgumentException. Unchecked exceptions often indicate programming errors or invalid arguments.

4. try and catch

A try block contains code that may throw an exception. A matching catch block handles a particular exception type. More specific catch blocks should appear before broader ones. A program should catch exceptions it can meaningfully handle and should avoid silently ignoring failures.

Code: try-catch

```
try {
int result = 10 / 0;
System.out.println(result);
} catch (ArithmeticException e) {
System.out.println("Cannot divide by zero");
}
```

5. finally

A finally block is used for cleanup code that should run after a try/catch sequence, whether or not an exception occurred, subject to abrupt JVM termination and other exceptional circumstances. For resources such as files, try-with-resources is usually preferred because it closes resources automatically.

6. throw and throws

The throw statement explicitly throws an exception object. The throws clause in a method declaration lists exception types that the method may pass to its caller. Use throw to raise a problem at a particular point; use throws to communicate a method's exception contract.

7. Custom exceptions

An application can define its own exception class when a domain-specific failure deserves a clear name. Extend Exception for a checked custom exception or RuntimeException for an unchecked one. Include a useful message and, when appropriate, preserve the original cause.

Code: custom exception

```
class InvalidAgeException extends Exception {
InvalidAgeException(String message) {
super(message);
}
}

class Voter {
static void checkAge(int age) throws InvalidAgeException {
if (age < 18) {
throw new InvalidAgeException("Age must be 18 or above");
}
}
}
```

8. Try-with-resources

Try-with-resources works with objects that implement AutoCloseable. Resources declared in the parentheses are closed automatically when the statement completes. This reduces boilerplate and helps prevent resource leaks. Multiple resources can be declared and are closed in reverse order of initialization.

9. Exception propagation

If a method does not handle an exception, the exception propagates to its caller. The call stack is unwound until a matching handler is found. If no handler is found, the thread's uncaught-exception handling mechanism receives the failure. A clear exception boundary helps an application report meaningful errors.

10. Best practices

Catch the most specific exception that can be handled. Do not use exceptions for ordinary control flow. Do not expose sensitive information in error messages. Preserve causes when wrapping exceptions. Log enough context to diagnose a failure, but avoid logging secrets. Validate inputs at system boundaries.

11. Common examples

NumberFormatException may occur when parsing text that is not a valid number. ArrayIndexOutOfBoundsException can occur when an array index is outside its valid range. FileNotFoundException can occur when a requested file cannot be opened. NullPointerException can occur when code attempts to use a null reference in a way that requires an object.

12. Review questions

1. What is the difference between checked and unchecked exceptions?

2. What does finally do?

3. Explain throw versus throws.

4. Why is try-with-resources useful?

5. How can a custom exception be created?